

POO

PA – 18241

Programação Aplicada

Ricardo Marau
marau@ua.pt



ESTGA

universidade de aveiro

Nesta aula

- POO – Conceitos
- Encapsulamento
- Construtores/Inicializadores
- Herança

Conceitos

- ◉ Linguagens procedimentais:
 - ◉ Criar estruturas de dados e Alocação em memória.
 - ◉ Criar procedimentos para manipular esses dados.
- ◉ Programação Orientada a Objetos
 - ◉ Instanciar objectos que podem conter instanciação de dados com procedimentos específicos para manipular esses dados.
 - ◉ Tudo é agrupados em objectos auto-suficientes.
 - ◉ Re-usabilidade a vários níveis

Carro
nome marca cor motor
acelera() trava()

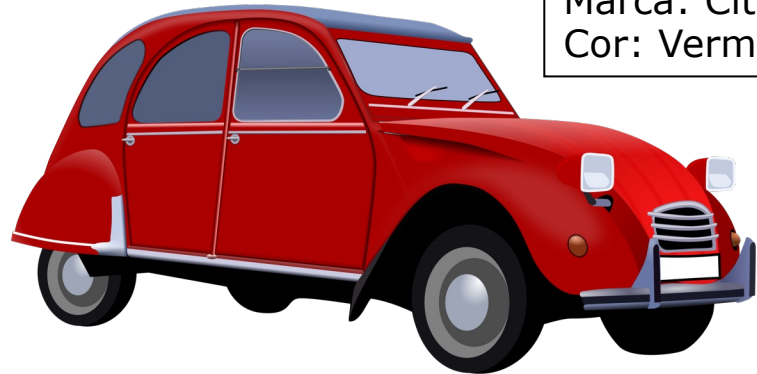
Nome da Classe

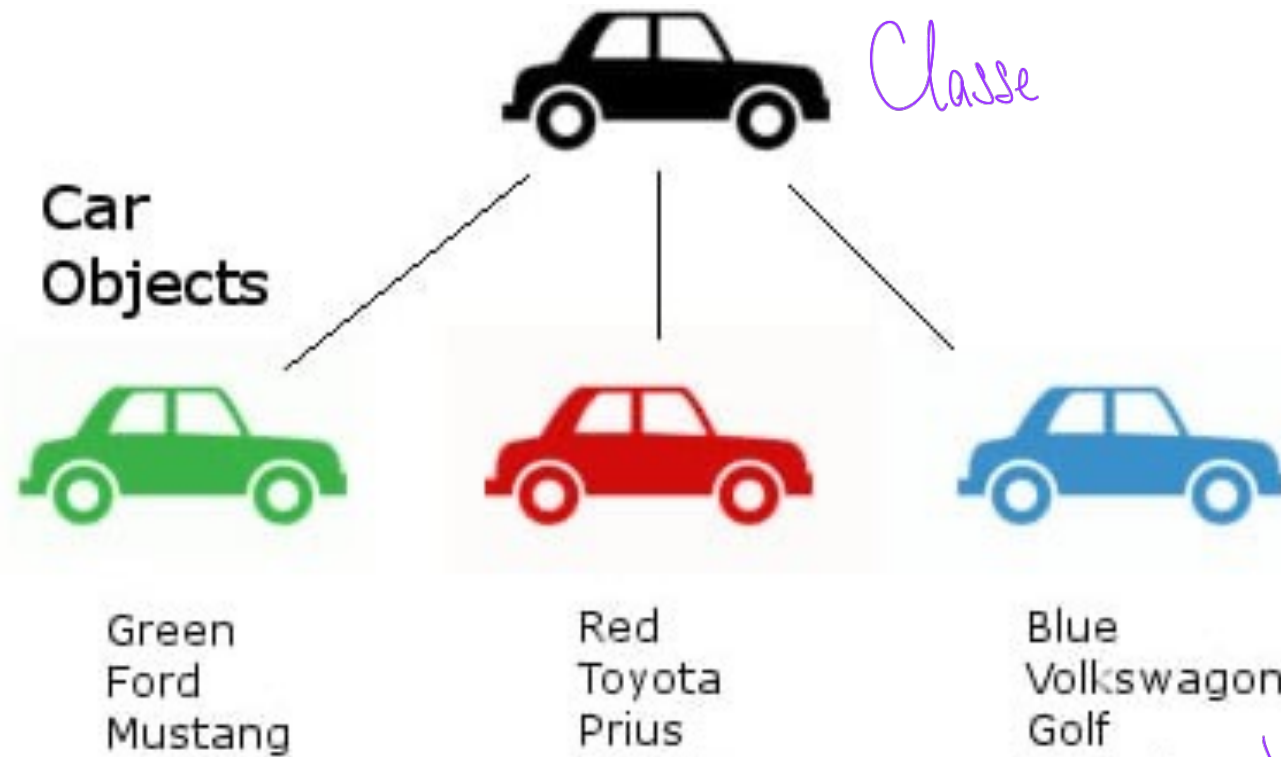
Atributos
*Características
que identificam os
objetos*

Métodos

Objeto

Nome: 2CV
Marca: Citroën
Cor: Vermelho





obj
a b c d
obj = Val4(1, 2, 3, 4)
obj.media()

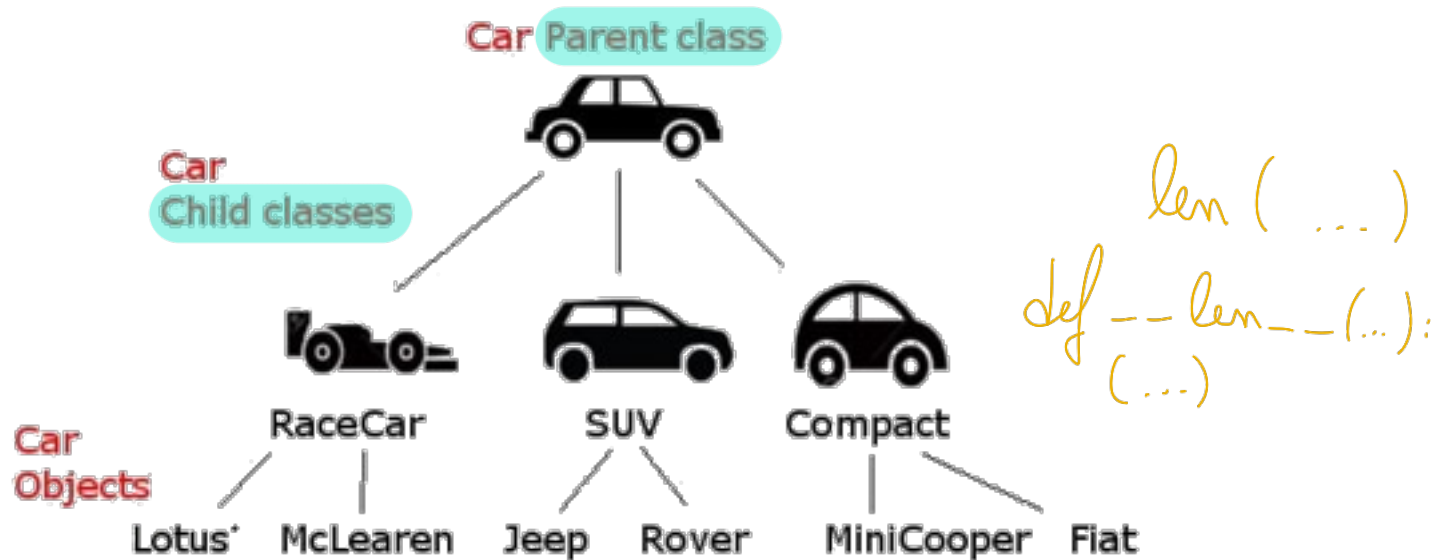
Val4
a = 1
b = 2
c = 3
d = 4

calcular media
adicionar valor

- Classe *↳ Blueprint / Modelo*
 - Protótipo onde se insere um conjunto de objectos.
 - Não armazena propriedade/atributos dos objectos.
- Objecto
 - Instanciação/Materialização do conceito - Classe.
 - Normalmente em associação ao mundo real que se pretende modelar. (Nome)
 - Em código, normalmente, haverá uma variável a “apontar” para o objeto
- Método
 - Ação executada por um objecto. (Verbo)
- Campo ou atributo
 - Característica de um determinado objecto.

Herança

- Mecanismo pelo qual uma classe (**sub-classe**) pode incorporar (herdar de) uma outra classe (**super-classe**), aproveitando atributos e métodos.



Polimorfismo

- Mecanismo pelo qual **um método herdado pode-se comportar de forma diferente em classes diferentes**.
- Exemplo: Ferrari e 2CV “acelera”m e “trava”m de forma diferente.

Class vs Objecto

- Classe define atributos e métodos da classe
 - nomes e verbos
- Objecto é a instânciação de uma classe.

class Pessoa(): —————> Definição
pass

pessoa_1 = Pessoa() ← Instânciação de objeto

↑
variável

↑
atribuição

look!
referência (nome da variável)

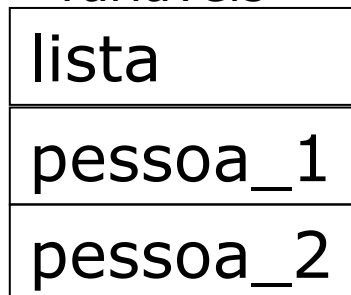
Instância - Referências

variável *objeto*
lista = []

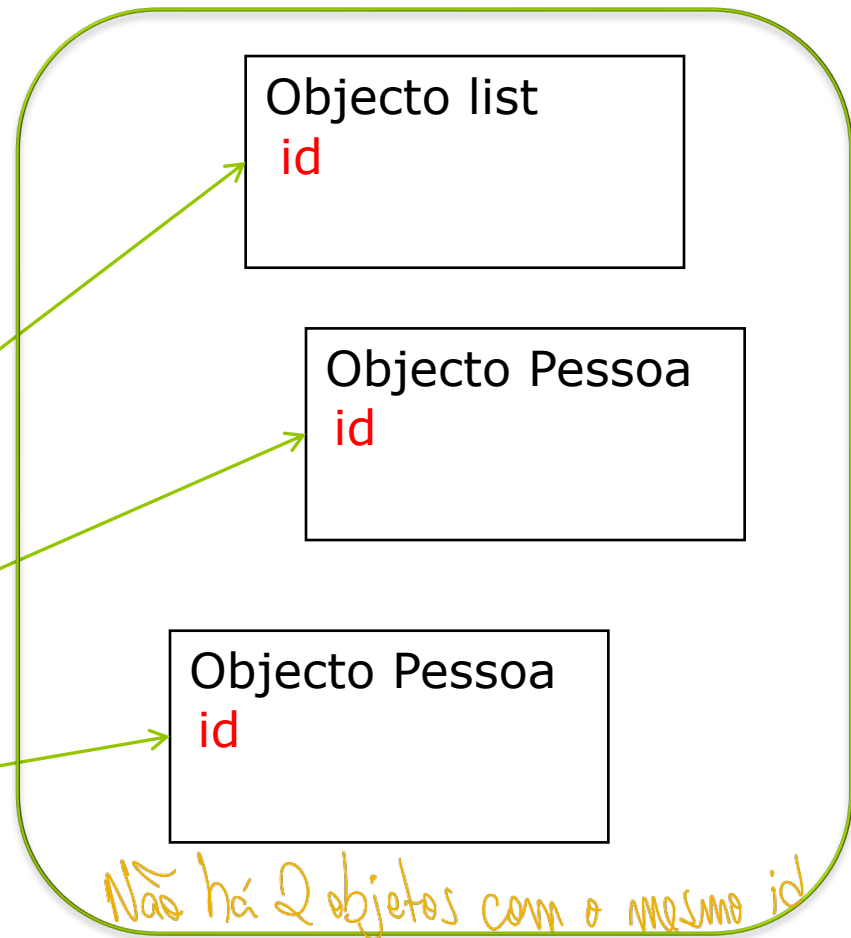
```
class Pessoa():  
    pass
```

variáveis *objetos*
pessoa_1 = Pessoa()
pessoa_2 = Pessoa()

Memória simplif.
<variáveis>



Memória simplif.
<pool>



id é uma característica do objeto

...

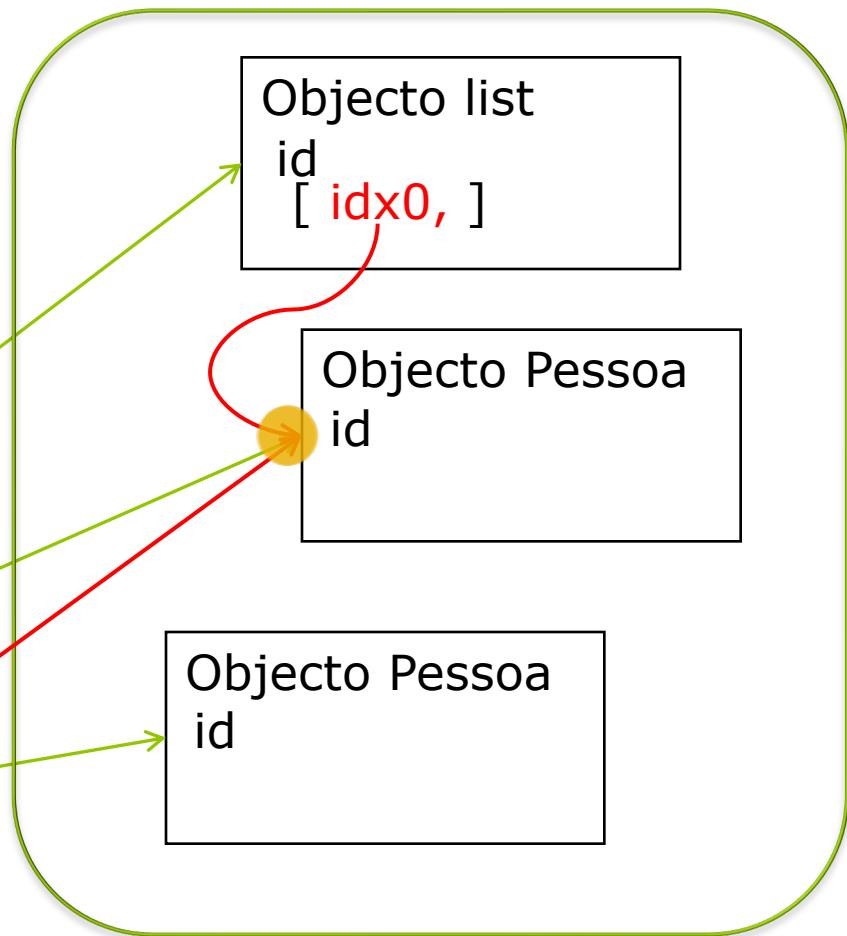
Ⓐ aponta para o mesmo id,
a = pessoa_1 aponta para
o mesmo objeto

lista.append(a)

Memória simplif.
<variáveis>

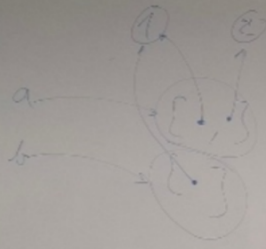
lista
pessoa_1
pessoa_2
a

Memória simplif.
<pool>



$a.append()$

$a == b$



def getroffen(obj): b
=:

return aus obj auf om. obj

"Pech" == "Pech"
is

Métodos

```
class Pessoa():  
    def fala(self):  
        print ('Sou um objecto Pessoa id:', id(self))  
        print ('Aka:', self.nome)
```

self

variável que aponta para o objeto em questão, simboliza cada objeto individualmente.

argumento da função

↳ Also known as

```
peessoa = Pessoa()  
peessoa.nome = "Trump"  
peessoa.fala()  
Pessoa.fala() ❌
```

TypeError: fala() missing 1 required positional argument: 'self'

- Através de **self** é possível aceder a todos os **membros** e **atributos** associados (**bounded**) ao objecto instanciado.

Pessoa.fala(peessoa) ~> já funciona

Atributos

*l.append(10)
list.append(l, 10)*

- Variáveis intrínsecas a um objecto!

```
class Pessoa():  
    pass
```

```
manel = Pessoa()  
manel.nome = "Manuel"  
manel.idade = 40
```

```
print(manel.nome)  
print(manel.__dict__)
```

Atributos do objecto
referenciado por
manel

*criação de atributos
externamente ao objeto (não
recomendado)*

```
Manuel  
{'nome': 'Manuel', 'idade': 40}
```

Construtor

- Em Python há o método `__init__()`
 - aka Método **construtor**
 - que na verdade não é construtor, mas **inicializador**.
 - O método `__new__()` é que trata da **construção**.
 - Útil em cenários avançados.

```
class Pessoa():
```

```
    def __init__(self, nome, idade):
```

```
        self.nome = nome
```

```
        self.idade = idade
```

Inicialização
dos atributos

```
manel = Pessoa("Manuel", 40)
```

Documentários (Documentação de código / classes)

docstring

```
class Pessoa():
```

```
    """ Pessoa encapsula um nome e uma idade """
```

```
    def __init__(self, nome, idade):
```

```
        """
```

```
        Inicializa um novo objeto Pessoa (descrição da função)
```

```
        :param nome: o nome da pessoa
```

```
        :param idade: a idade da pessoa
```

```
        :returns nothing """
```

```
        self.nome = nome
```

```
        self.idade = idade
```

```
help(Pessoa)
```

@class method

cls → métodos de classe

String multi-linha : s = """

*ou
apóstrofes*

```
"""
```

@static method
def ... (...):

→ não tem cls nem self como 1º argumento

Herança - Inheritance

atributo de classe

- Permite que se crie primeiro uma classe geral (superclasse) que depois é “extendida” para uma mais específica (subclasse).
- A subclasse herda o acesso aos atributos e métodos e ainda permite adicionar atributos e métodos.
 - Apenas os não privados.


```
class Veiculo: ← Classe Super
    pass
```

```
class Carro(Veiculo): ← Sub Classe
    pass
```

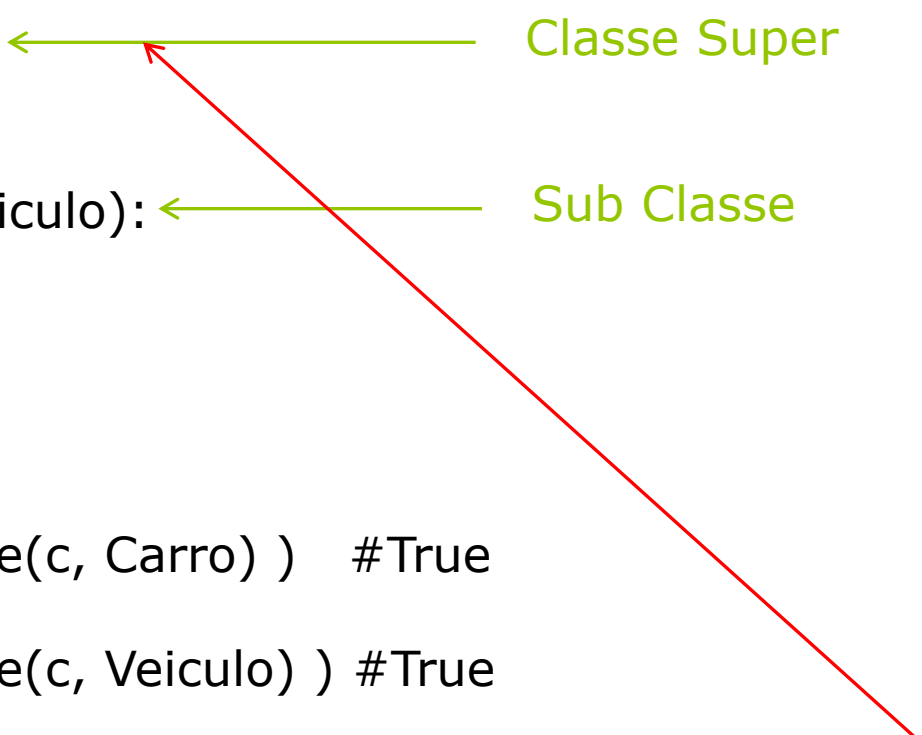
```
c = Carro()
```

```
print( isinstance(c, Carro) ) #True
```

```
print( isinstance(c, Veiculo) ) #True
```

```
print( isinstance(c, object) ) #True
```

```
class Veiculo(object):
    pass
```



nome Variável

↳ significa que é um atributo ou método privado

super() - refere-se à classe Pai

Métodos herdados
Públicos

```
class Carro(Veiculo):
```

```
    def __init__(self, nome, cor, modelo):
        super().__init__(nome, cor)
        self.__modelo = modelo
```

```
    def getDescricao(self):
        return self.getNome() + " " + self.__modelo + " " + self.getCor()
```

```
c = Carro("Ford Mustang", "Vermelho", "GT350")
```

```
print(c.getDescricao())
print(c.getNome())
```

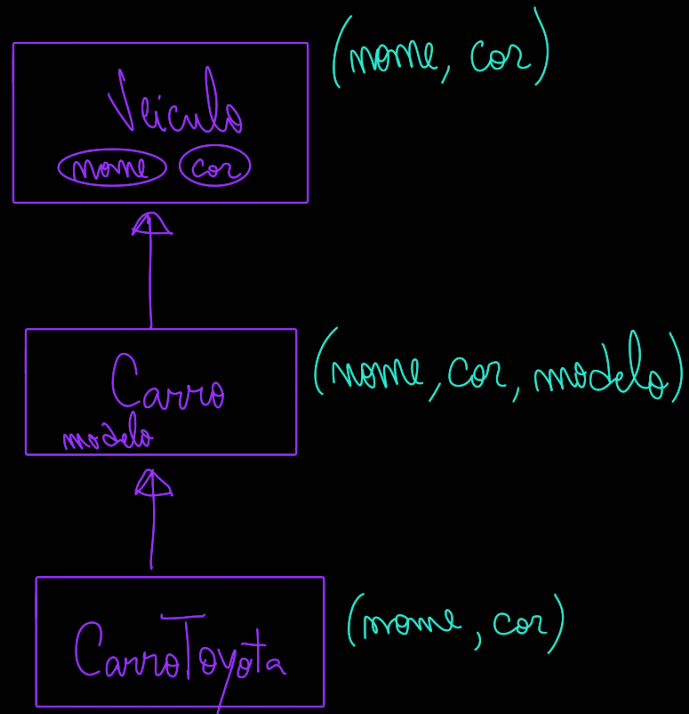
```
class Veiculo:
```

```
    def __init__(self, nome, cor):
        self.__nome = nome
        self.__cor = cor
```

```
    def getCor(self):
        return self.__cor
```

```
    def getNome(self):
        return self.__nome
```

```
    def anda(self):
        print("Veiculo a andar...")
```



```
class CarroToyota (Carro):  
    def __init__(self, nome, cor):  
        super().__init__(nome, cor, "Toyota")
```

Override

```
class Carro(Veiculo):
```

```
    def __init__(self, nome, cor, modelo):  
        super().__init__(nome, cor)  
        self.__modelo = modelo
```

```
    def anda(self): #Overriding  
        super().anda()  
        print("Vruuuuum...")
```

```
c = Carro("Ford Mustang", "Vermelho", "GT350")  
c.anda()
```

```
class Veiculo:
```

```
    def __init__(self, nome, cor):  
        self.__nome = nome  
        self.__cor = cor
```

```
    def getCor(self):  
        return self.__cor
```

```
    def getNome(self):  
        return self.__nome
```

```
    def anda(self):  
        print("Veiculo a andar...")
```

{ Neste caso imprime duas frases }

Override – object methods

- O object é a classe raiz em python
 - Mas já possuí alguns métodos nativos

```
class Veiculo:  
    pass
```

```
v=Veiculo()
```

```
print(v)
```

```
#Imprime: <__main__.Veiculo object at 0x10f6923c8>
```

```
print (dir(v))
```

```
#Imprime: ['__doc__', '__str__', '__eq__', '__init__', ... ]
```

Retorna uma string representativa do objeto

--str-- descreve/representa o objeto

`l = []`
`l = list()`) same

`print(l)` ~> `printa []` /n

`print()` executa o *--str--* / ou *--rep--* /
usa dependendo do objeto

class Veiculo:

#Override de __str__()

def __str__(self):

return "Sou um veiculo da classe Veiculo"

escondido, é o `print()` função que põe strings formatadas, geralmente usa o *--rep--*

`v=Veiculo()`

`print(v)`

#Imprime: Sou um veiculo da classe Veiculo

A função `print` recorre ao `__str__()`
Para imprimir o conteúdo do objecto

Polimorfismo - Classes Abstract

class VeiculoAbstrato:

```
def __init__(self, nome, cor):  
    self.__nome = nome  
    self.__cor = cor
```

```
def getCor(self):  
    return self.__cor
```

```
def getNome(self):  
    return self.__nome
```

```
def __str__(self):  
    return self.getDescricao()
```

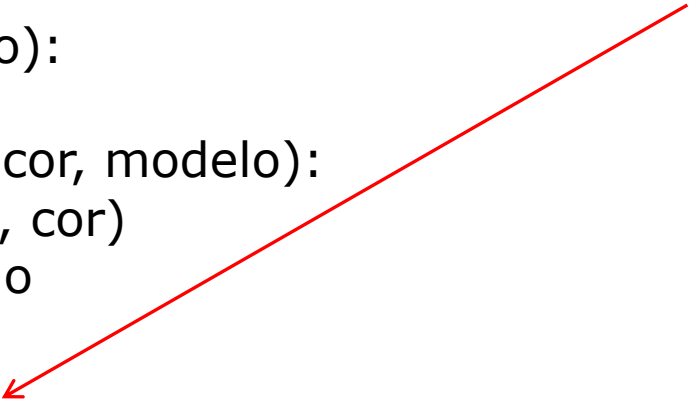
```
def getDescricao(self):  
    raise NotImplementedError("Subclass tem de implementar getDescricao()")
```

a classe só é abstrata por causa deste raise
Pode estar, ou não.

↳ Forçar crashar o programa

Criar um "erro"/problema que um programador vai ter que tratar/corrigir/implementar

Implementação do método
Abstrato



```
class Carro(VeiculoAbstrato):

    def __init__(self, nome, cor, modelo):
        super().__init__(nome, cor)
        self.__modelo = modelo

    def getDescricao(self):
        return self.getNome() + " " + self.__modelo + " " + self.getCor()

c = Carro("Ford Mustang", "Vermelho", "GT350")
print(c)
#Imprime Ford Mustang GT350 Vermelho
```


- Python não possui classes abstratas nativamente.
- O que vimos foi uma forma de imitar o funcionamento abstrato.
- Há ferramentas mais avançadas para gerir as classes abstratas em Python:

(**from abc import** **ABC**, **abstractmethod**)

class AbstractClass(ABC):
 @abstractmethod
 def foo(self):
 pass

class SubClasse(AbstractClass):
 pass

t=SubClasse()



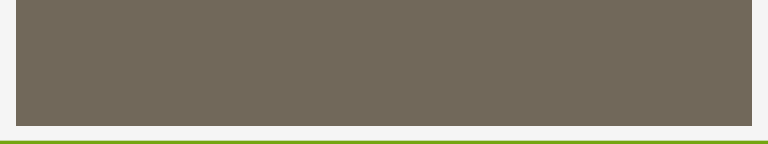
→ Abstract Class

→ assim não é preciso o raise

Imprime:

TypeError: Can't instantiate abstract class SubClasse with abstract methods foo

Veiculo(ABC)



Fim

- Questões?

